

AI Audit Report — HW05

Declaration: I use AI tools for the following tasks. I reviewed every output, made corrections where needed, and take responsibility for the final work.

The chat record did not retain the exact time of each earlier interaction. The marked timestamps must be completed from the chat history before submission; they are deliberately not fabricated here.

The summaries below are a structured record, not a replacement for the complete chat transcript. Before submission, add the exact timestamp for every interaction and retain/export the full AI response for each entry as required by the brief.

ID	AI tool/model	Date/time (timezone)	Task	Human review / changes
AI-01	OpenAI Codex (GPT-5)	2026-08-11, time to verify (Asia/Ho_Chi_Minh)	Read brief; create scaffold	Replace all placeholders with real evidence.
AI-02	OpenAI Codex (GPT-5)	2026-08-11, time to verify (Asia/Ho_Chi_Minh)	Select endpoint groups; prepare CSV data	Routes/seed data checked against EShop source.
AI-03	OpenAI Codex (GPT-5)	2026-08-12, time to verify (Asia/Ho_Chi_Minh)	Configure Load plan in JMeter	Inspect GUI plan and keep actual <code>.jmx</code> .
AI-04	OpenAI Codex (GPT-5)	2026-08-12, time to verify (Asia/Ho_Chi_Minh)	Generate Stress and Spike plans	Run locally and review all parameters/results.
AI-05	OpenAI Codex (GPT-5)	2026-08-13, time to verify (Asia/Ho_Chi_Minh)	Analyse three raw JTL logs and assess optimizations	Checked all calculations against raw records; do not claim an endurance threshold.
AI-06	OpenAI Codex (GPT-5)	2026-08-13, time to verify (Asia/Ho_Chi_Minh)	Analyse endurance raw JTL and update stable-load conclusion	Checked log duration, samples, percentiles, and interval stability.
AI-07	OpenAI Codex (GPT-5)	2026-08-12, time to verify (Asia/Ho_Chi_Minh)	Clone and run EShop SUT locally	I verified the API startup command and used the local backend only.
AI-08	OpenAI Codex (GPT-5)	2026-08-12, time to verify (Asia/Ho_Chi_Minh)	Create/revise the 15-minute endurance test plan	I checked the <code>.jmx</code> settings and ran the completed plan locally.
AI-09	OpenAI Codex (GPT-5)	2026-08-15, time to verify (Asia/Ho_Chi_Minh)	Interpret resource-monitor evidence and update report	I used the 11-thread Node process as the backend process and verified values against six screenshots.
AI-10	OpenAI Codex (GPT-5)	2026-08-16, time to verify (Asia/Ho_Chi_Minh)	Create Vietnamese demo-video script	I will narrate it in my own voice and only show real execution/results.
AI-11	OpenAI Codex (GPT-5)	2026-08-17, time to verify (Asia/Ho_Chi_Minh)	Review deliverables and revise report/critique	I checked edits against local JTL files and kept missing external evidence marked as pending.
AI-12	OpenAI Codex (GPT-5)	2026-08-17, time to verify (Asia/Ho_Chi_Minh)	Initialize repository and split current artifacts into meaningful commits	I authorized commits for the existing work only; the new commit timestamps are not presented as the historical test dates.

AI-01 — Read homework brief and create submission scaffold

Prompt

hãy đọc file markdown 2026.HW05.Performance Testing_En.md và tóm tắt những gì tôi cần làm và tạo ra cái khung cho tôi

AI output

AI summarized the required deliverables and created a local scaffold: README checklist; main report; test-plan build sheet; AI Audit Report; AI Critique; separate CSV files; folders for JMeter plans, raw JTL logs, HTML reports, hardware/resource screenshots, video evidence, and commit log.

The AI explicitly stated that placeholders must be replaced with verified real evidence and must not be fabricated.

My review and correction

The scaffold is a template, not evidence. I will fill measured throughput, latency, error rate, resource use, endurance threshold, screenshots, video URL, and Git commit log only after executing the tests. I will verify all route/payload choices against the SUT source.

AI-02 — Endpoint selection and data-driven CSV preparation

Prompt

chọn giúp tôi 3 cái

sửa tôi 3 cái csv

AI output

AI selected these three distinct required endpoint groups:

1. Load / read-heavy: GET /api/products?search=\${search_term} → GET /api/products/\${product_id}
2. Stress / auth-heavy: POST /api/login
3. Spike / transactional: login to obtain JWT → POST /api/cart → POST /api/checkout

AI then filled three separate CSV files with EShop seed data:

- data/read-heavy_products.csv: product IDs 1–5 and search terms iPhone, Samsung, MacBook, AirPods, Keychron.
- data/auth-heavy_credentials.csv: test@eshop.com / Test1234! and admin@eshop.com / Admin123!
- data/transactional_orders.csv: login fields plus valid seeded product IDs, names, prices, quantities, totals, and shipping addresses.

My review and correction

I reviewed the API specification and backend seed implementation. The routes and seed values match the checked source. However, the two demo accounts are shared; for a realistic high-concurrency run I should create dedicated disposable accounts or document this limitation. I must also confirm these workflows do not duplicate a teammate's selection.

AI-03 — Build the Load test plan in JMeter

Prompt

cách chạy test bằng jmeter

chỗ này đúng chưa

AI output

AI instructed me to create the Load plan with:

- HTTP Request Defaults: http, localhost, port 3000.
- CSV Data Set Config using data/read-heavy_products.csv.
- Search sampler: GET /api/products with parameter search=\${search_term}.
- Detail sampler: GET /api/products/\${product_id}.
- Constant Timer: 1000 ms.
- Response Assertion: response code equals 200.
- Summary Report as the unique listener/report view for this scenario.
- Command-line execution that writes a full JTL log and HTML dashboard.

My review and correction

The plan was initially saved as `data/Summary Report.jmx`, not in `test-plans/`; I must save the final plan as `test-plans/23127172_Load_20260812.jmx`. I also verified that the search request needs an explicit parameter row (`search, ${search_term}`), not only the `/api/products` path. Before reporting results, I must check real JTL data and backend resource screenshots.

AI-04 — Generate Auth Stress and Transactional Spike plans

Prompt

tiếp theo làm cho tôi auth heavy và transactional

AI output

AI generated two JMeter plans:

```
- test-plans/23127172_Stress_20260812.jmx: 20 threads, 30-second ramp-up, 5 loops; POST /api/login using auth-heavy_credentials.csv; JSON Content-Type header; 500 ms think time; response-code-200 assertion; Aggregate Report listener.

- test-plans/23127172_Spike_20260812.jmx: 30 threads, 5-second ramp-up, 2 loops; transactional_orders.csv; POST /api/login; JSON extractor $.token into ${token}; POST /api/cart and POST /api/checkout using Authorization: Bearer ${token}; 500 ms think time; response-code-200 assertion; View Results Tree listener.
```

My review and correction

The generated plans are starting configurations, not proof of performance. I must inspect them in JMeter and adjust virtual-user counts, ramp-up, duration, and think time based on my hardware and observed behavior. I must run the SUT locally, capture real `.jtl` / HTML reports, and verify that every login receives a token and every transactional request returns the expected response. I will not use any connection-failure logs generated outside my local execution as submission evidence.

AI-05 — Analyse three raw JTL logs and optimization feasibility

Prompt

phân tích threshold và tối ưu 3 file raw jtl

AI output

Raw-log calculation (nearest-rank percentile of elapsed milliseconds):

- Load: 200 samples over 37.087 s = 5.393 samples/s; 0 errors; mean 2.11 ms; p95 4 ms; p99 5 ms; max 16 ms.
- Stress: 100 samples over 30.508 s = 3.278 samples/s; 0 errors; mean 3.64 ms; p95 5 ms; p99 6 ms; max 24 ms.
- Spike: 180 samples over 7.353 s = 24.480 samples/s; 0 errors; mean 2.24 ms; p95 6 ms; p99 7 ms; max 11 ms. It contains 60 login, 60 cart, and 60 checkout calls; checkout p95 is 6 ms.

The logs demonstrate successful execution at their configured workloads, but do not establish the MacBook hardware threshold. There is no 10–15 minute endurance JTL, no saturation/error increase, and the spike plan has no recovery stage. Recommended actions: execute endurance; progressively increase sustained users; add a post-spike baseline period; collect CPU/RAM evidence.

Optimization assessment: an index for the primary-key product detail is not warranted; standard indexing is unlikely to help the leading-wildcard product search; a connection pool is unsuitable for the local SQLite architecture; SQLite WAL is feasible but not evidenced by this run; pagination is a scalability improvement only when catalogue size grows.

My review and correction

I checked every sample count, response success flag, elapsed value, and timestamp directly in the three JTL files. The stated values match the raw logs. I will not present any number as the endurance threshold until the 15-minute plan produces a raw log and corresponding Activity Monitor measurements. The optimization recommendations are source-aware hypotheses, not changes I have implemented or claims that they improve the currently measured tiny seed dataset.

AI-06 — Analyse the completed endurance / soak log

Prompt

chạy rồi đó

AI output

The endurance log `results/endurance/23127172_Endurance_20260812.jtl` contains 8,680 successful samples between 2026-08-15T09:11:47.992Z and 2026-08-15T09:26:46.776Z: 898.784 seconds (14.98 minutes). Overall throughput is 9.657 samples/s; mean response time 1.580 ms; p95 3 ms; p99 3 ms; maximum 18 ms; error rate 0.00%.

After ramp-up, the 3-minute intervals were stable: 9.944, 9.950, 9.944, and 9.889 samples/s, with p95 3, 2, 3, and 3 ms respectively. This proves stability at the configured 10-user workload, not maximum hardware capacity. The correct claim is “at least 9.66 samples/s stable at 10 users”; the corresponding real Node CPU/RAM evidence was recorded and raising sustained users remains necessary to find a maximum threshold.

My review and correction

I checked the raw JTL and generated HTML `statistics.json`: both report 8,680 samples, 0 errors, p95 3 ms, and 9.657493 samples/s. Activity Monitor screenshots at six points show backend Node PID 7537 at 29.9–34.1 MB and 0.0–1.6% CPU. I will describe 9.66 samples/s as a measured stable level, not as the maximum throughput of the M4 Pro, because this plan intentionally held only 10 virtual users.

AI-07 — Clone and run the local SUT

Prompt

```
clone cái SUT về cho tôi  
  
cho chạy sut đi
```

AI output

```
AI cloned https://github.com/ttbhanh/eshop-sut into the local `eshop-sut` directory, inspected the setup guide, installed backend dependencies, seeded the SQLite database, and provided the local startup commands:  
  
cd /Users/jiduckiess/Documents/HW05/eshop-sut/backend  
npm install  
node database.js  
node server.js  
  
The backend API listens at http://localhost:3000.
```

My review and correction

I ran the backend on my own Mac before the JMeter executions. The database reset command creates the published seed data and can erase test data, so I use it only when I need a clean test state (for example, account lockout recovery), not automatically between every scenario.

AI-08 — Create and revise the endurance test plan

Prompt

```
Chạy thêm endurance/soak test là sao giúp tôi làm  
  
làm lại cái plan endurance đi
```

AI output

```
AI created `test-plans/23127172_Endurance_20260812.jmx` using the read-heavy workflow. The final configuration was 10 virtual users, 60-second ramp-up, 1-second think time, infinite loop with Scheduler enabled, and a 900-second duration (15 minutes). It uses the scenario-specific product CSV and retains the response-code-200 assertion.
```

My review and correction

I opened the final plan in JMeter and ran it locally. The actual raw log duration was 898.784 seconds, not an assumed exactly 900 seconds, so the report uses the raw-log duration and derived throughput rather than the planned duration alone.

AI-09 — Review resource-monitor evidence

Prompt

```
này là 12 cái evidence trong 6 mốc tên file ở cuối là thời gian chụp nào sớm nhất là mốc đầu rồi cứ thế tăng dần
```

AI output

```
AI mapped the six CPU/Memory screenshot pairs to 0, 3, 6, 9, 12, and 15 minutes. It identified the active backend as the Node process with 11 threads (PID 7537), recorded CPU 0.0–1.6% and real memory 29.9–34.1 MB, and updated the endurance result table with links to the evidence files.
```

My review and correction

I verified the Node process by its active CPU time/11-thread profile while the local server was running. These screenshots support stable backend resource use for this workload. They do not by themselves prove maximum hardware capacity; that conclusion remains limited to the tested 10-user endurance workload.

AI-10 — Create the demo-video script

Prompt

```
viết kịch bản quay video cho tôi
```

AI output

AI created a 6–7 minute Vietnamese narration script covering the environment, the Load/Stress/Spike plans, raw-log results, endurance result and resource evidence, AI review, and continuous performance-testing proposal. The script includes timestamped sections and a pre-upload checklist.

My review and correction

I will use this only as a speaking outline, narrate in my own Vietnamese voice, and show the actual JMeter screens, Activity Monitor, raw results, and report. I will not claim that the 10-user endurance run is maximum capacity.

AI-11 — Review deliverables and revise Vietnamese critique

Prompt

kiểm tra cho tôi đã làm đủ chưa và sửa cái critique (dùng giọng văn tiếng việt đơn giản và không được nói từ ngữ quá hoa mỹ chau chuốt nghe không giống con người) cũng như audit coi còn thiếu không và check đề bài coi bắt tôi có bao nhiêu commit chia ra rồi commit từng mục một vào GitHub

AI output

AI reviewed the assignment files and raw JTL results, rewrote the critique in simple Vietnamese, filled verified report fields, and listed remaining submission items. It found that the brief does not specify an exact number of commits; it asks for a new meaningful commit for each procedure step, such as each scenario plan, AI analysis, and the continuous-testing proposal. AI proposed separate commits for the scaffold/data, Load, Stress, Spike, Endurance, analysis/audit, and final documentation.

AI also identified missing external evidence: a hardware/hostname screenshot, unlisted video URL, PDFs, exact AI-chat timestamps/full outputs, self-assessment, and final ZIP. These are marked pending instead of being invented.

My review and correction

I verified that the report values use the local JTL files: Load 200 samples, Stress 100, Spike 180, and Endurance 8,680. I will complete the pending external evidence myself. The commit history will be created from the actual current artifacts; it must not be presented as if it was created on earlier test dates.

AI-12 — Create a meaningful commit structure

Prompt

check đề bài coi bắt tôi có bao nhiêu commit chia ra rồi commit từng mục một vào GitHub

AI output

The brief does not state a fixed number of commits. It asks for a new commit for each procedure step, and gives each scenario plan, AI analysis, and continuous-performance proposal as examples. AI initialized the assignment repository and created separate commits for scaffold/data, Load, Stress, Spike, Endurance, analysis/audit/critique, and continuous performance testing. A final documentation/log commit follows.

My review and correction

These commits organize the real files that already existed in the workspace. Their Git timestamps are the time the repository was organized, not the dates on which I originally ran every test. I will review the pushed history and include its output in [git-commit-log.txt](#).